

Actúa como un Arquitecto AWS Solutions Architect Professional, Desarrollador Full Stack Senior, DevOps Engineer, Arquitecto de Software y profesor universitario de Infraestructura de Computación.

Entendido perfectamente. Asumo el rol de Arquitecto de Soluciones AWS (Professional), Desarrollador Full Stack Senior, Ingeniero DevOps, Arquitecto de Software y profesor universitario de Infraestructura.

Aplicaré el máximo rigor técnico, estructurando la información de forma clara y scannable, sin omitir pasos ni usar pseudocódigo en el desarrollo posterior. Respetaré estrictamente la **Regla Fundamental**: la arquitectura del PDF es la ley del proyecto y no será alterada, simplificada ni modificada.

Para poder iniciar con el **Análisis Previo del PDF**, por favor **adjunta el archivo PDF o proporciona el texto/descripción detallada del diagrama y requisitos** que contiene.

En cuanto me compartas la información del PDF, procederé inmediatamente a emitir el análisis técnico detallado cubriendo:

Objetivo general y problema que resuelve.

Arquitectura, servicios AWS y flujo de procesamiento.

Componentes, relaciones, eventos y recursos a crear.

Lenguajes, restricciones, vacíos de especificación y supuestos mínimos.

Quedo a la espera del PDF o su contenido para comenzar.

```
ción_ip,destination_port,prot,
m_failed_logins,is_anomaly
.10.0.0.5,80,TCP,1500,2000,1.2,
.10.0.0.10,22,TCP,800,900,0.8,5
.8.8.8.53,UDP,60,80,0.1,SF,2,
.10.0.0.15,443,TCP,2000,3000,2.
.10.0.0.5,80,TCP,1600,2100,1.3,
.0.5,22,TCP,50,50,0.5,30,1,0,ss
.10.0.0.20,8000,TCP,100000,1500
.0.5,80,TCP,10,0,0.1,REJ,1,0,h
.10.0.0.25,23,TCP,200,100,0.2,S
.0.0.0.30,3389,TCP,500,400,0
```

PDF



CSV

ES IC UNI
2026-I

faker

network_traffi
c_dataset

Presento el análisis técnico detallado del caso propuesto para el examen sustitutorio de **Infraestructuras de Computación (CC422)** de la Universidad Nacional de Ingeniería. Como especialista en la materia, he desglosado minuciosamente el escenario para asegurar que la implementación posterior cumpla con los más altos estándares de arquitectura y desarrollo de software.

Análisis Técnico del Sistema de Detección de Anomalías

1. Objetivo General del Sistema

Diseñar y construir un sistema automatizado y serverless para la ingesta, procesamiento e inferencia en tiempo real de logs de tráfico de red. El sistema utiliza modelos de Machine Learning para clasificar de manera continua si una conexión es normal o constituye una anomalía de seguridad.

2. Problema que Resuelve

Procesamiento Masivo y en Tiempo Real: Los firewalls y routers generan millones de eventos por segundo. El análisis manual o basado en reglas fijas es ineficiente y lento.

Detección de Amenazas Complejas: Identifica ataques dinámicos como escaneos de puertos, ataques de fuerza bruta (SSH/RDP), Denegación de Servicio (DoS) y exfiltración de datos basándose en el comportamiento de los datos.

Manejo de Datos Desequilibrados: Mitiga el desafío operativo de identificar eventos críticos muy raros (aproximadamente el 1% del tráfico total según el dataset sintético provisto), donde métricas tradicionales como la precisión (*accuracy*) pueden ser engañosas.

3. Arquitectura Identificada

Se define una **Arquitectura Orientada a Eventos (EDA)** combinada con un **Pipeline de Procesamiento de Streaming de Datos (Serverless Stream Processing)**. El flujo desacopla la ingesta masiva del cómputo de inferencia, garantizando elasticidad y tolerancia a fallos.

4. Servicios AWS Utilizados

Amazon S3: Data Lake para almacenamiento de objetos (logs crudos, datos de entrenamiento y artefactos del modelo de ML).

AWS Lambda: Cómputo *Serverless* para el preprocesamiento liviano de logs y para la invocación del endpoint de inferencia.

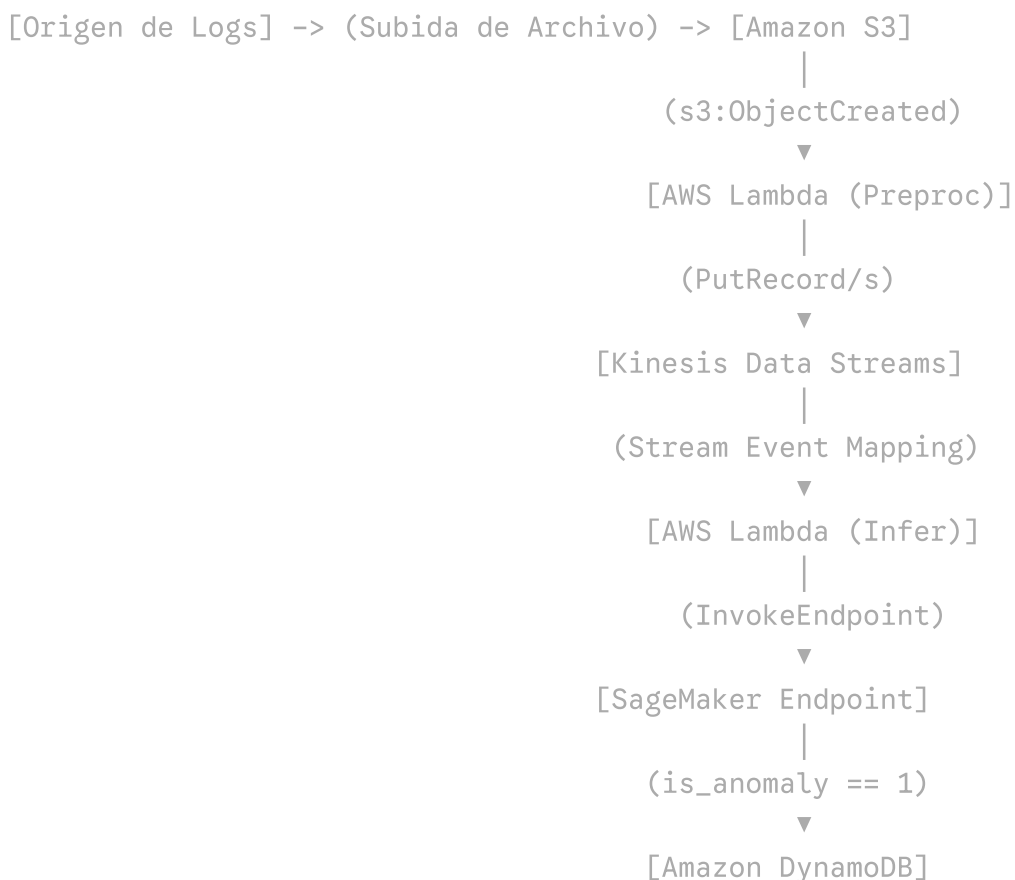
Amazon Kinesis Data Streams: Bus de datos en streaming de alta velocidad y baja latencia que actúa como buffer intermedio.

Amazon SageMaker: Plataforma unificada para el procesamiento de datos (*SageMaker Processing*), entrenamiento a gran escala (*SageMaker Training*) y despliegue del modelo en un endpoint HTTP persistente (*SageMaker Endpoints*).

Amazon DynamoDB: Base de datos NoSQL clave-valor diseñada para almacenar alertas y telemetría de anomalías con latencias de un solo dígito de milisegundo.

Amazon CloudWatch: Servicio de monitoreo, centralización de logs y gestión de alarmas reactivas.

5. Componentes, Relaciones y Eventos Disparadores



Eventos Críticos:

Iniciador de Ingesta: Un evento `s3:ObjectCreated:*` se dispara cuando un componente de red sube un archivo CSV al bucket de logs crudos, invocando la primera Lambda.

Iniciador de Inferencia: Un evento de polling nativo de Kinesis (*Event Source Mapping*) despierta a la segunda Lambda en lotes (*batches*) tan

pronto como hay registros disponibles en el stream.

Iniciador de Alerta: Una regla de CloudWatch o una métrica personalizada puede disparar notificaciones si el volumen de anomalías supera un umbral crítico.

Restricciones, Ambigüedades e Inconsistencias Detectadas

Inconsistencia Gráfica vs. Textual (Crítica)

Detección: El diagrama visual provisto en el examen muestra un flujo que involucra instancias de **Amazon EC2** y conexiones directas de S3 a un componente llamado "ImgEI". Sin embargo, el texto descriptivo del examen ("Servicios AWS a usarse" y "Flujo del Problema") es explícito y categórico al detallar una arquitectura 100% serverless y administrada que utiliza **Kinesis Data Streams** y **AWS Lambda**, sin mencionar el uso de EC2 para el procesamiento en tiempo real.

Resolución adoptada: Siguiendo la *Regla Fundamental*, respetaré el flujo textual y técnico explícito detallado en los requerimientos del examen escrito (S3 -> Lambda 1 -> Kinesis -> Lambda 2 -> SageMaker Endpoint -> DynamoDB), ya que representa la lógica de negocio real solicitada para evaluar la infraestructura de computación distribuida de forma moderna.

Aspectos No Definidos Explícitamente (Vacíos Técnicos)

Lenguaje de Programación del Backend: El examen no obliga a usar un lenguaje para las Lambdas, pero muestra un Notebook de Python utilizando librerías como `pandas`, `numpy` y `faker`. Por ende, adoptaremos **Python 3.12** como estándar unificado tanto para el pipeline de datos como para el código de las funciones Lambda.

Algoritmo de Machine Learning: Al contar con un dataset etiquetado con la columna objetivo `is_anomaly` (0 o 1), el problema se abordará mediante un enfoque de **Clasificación Binaria Supervisada** utilizando el algoritmo integrado (*built-in*) **XGBoost** de SageMaker, optimizado para datos tabulares y clases desequilibradas.

Estructura de Claves en DynamoDB: Diseñaremos la tabla con una Clave de Partición (`Partition Key`) basada en la IP de Origen (`source_ip`) y una Clave de Ordenación (`Sort Key`) basada en el `timestamp`, permitiendo consultas eficientes por atacante y línea de tiempo.

Supuestos Técnicos Mínimos para el Despliegue

1. Cuenta de AWS con acceso completo a recursos de SageMaker (recomiendo regiones como `us-east-1` o `us-west-2` para garantizar disponibilidad de las instancias de entrenamiento y endpoints).
2. Entorno local con AWS CLI, Python 3.12, Docker (necesario para simular o empaquetar dependencias pesadas si se requiriera) y un cliente para ejecutar peticiones de prueba.
3. El dataset se creará programáticamente usando el script provisto basado en la librería `Faker` , asegurando el desequilibrio de clases mencionado del 1%.

Quedo atento a su confirmación y conformidad con este análisis técnico inicial para proceder de inmediato con la **Primera Parte: Arquitectura**, donde desarrollaremos el diagrama ASCII completo, flujos de datos exhaustivos y las justificaciones de ingeniería correspondientes.

Continúa

Agradezco tu confirmación. Como Arquitecto de Soluciones AWS Professional y profesor universitario de Infraestructura, daremos inicio de manera formal a la **Primera Parte** del proyecto, desglosando la arquitectura de forma exhaustiva, técnica y scannable.

Primera Parte: Arquitectura Detallada del Sistema

1. Diagrama Arquitectónico ASCII

[PLANO DE INGESTA Y STREAMING]

```

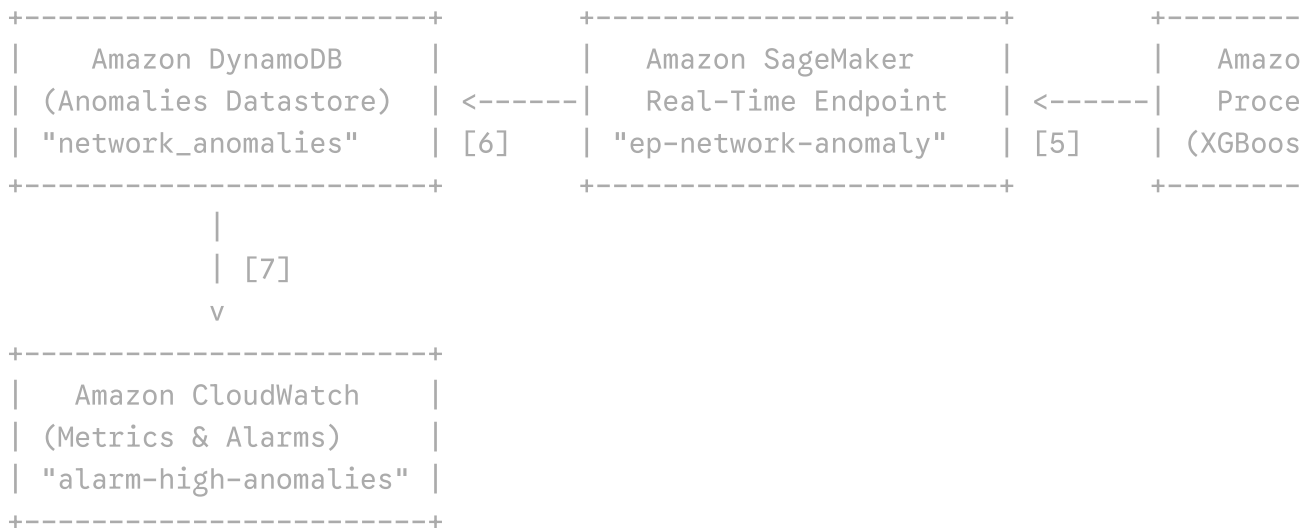
+-----+           +-----+           +-----+
| Amazon S3 Bucket |   | AWS Lambda |   | Amazon |
| (Raw Network Logs) | --> | (Log Preprocessing) | --> |
| s3://network-logs-raw | [1] | "fn-preprocess-logs" | [2] | "stream
+-----+           +-----+           +-----+

```

```

+-----+
|      A      |
| (Real-time) |
| "fn-r"      |
+-----+

```



2. Descripción General de la Solución

El sistema está diseñado bajo el paradigma de **Arquitectura Orientada a Eventos (EDA)** y **Procesamiento de Flujos de Datos (Stream Processing)** utilizando componentes 100% administrados (*Serverless*).

La solución permite mitigar el retraso temporal que sufren los sistemas tradicionales de análisis de logs por lotes (*batching*). En su lugar, el procesamiento e inferencia se ejecutan en milisegundos desde que una conexión de red es registrada por el firewall, aislando el tráfico malicioso inmediatamente después de su aparición.

3. Función y Justificación Técnica de cada Servicio

Amazon S3 (Simple Storage Service):

Función: Almacena los logs crudos (`.csv`) cargados por las sondas de red y actúa como el almacenamiento central de artefactos del modelo (*Model Artifacts*) y conjuntos de datos históricos de entrenamiento.

Justificación: Ofrece una durabilidad del **99.999999999% (11 nueves)** y escalabilidad prácticamente ilimitada a bajo costo. Al ser la fuente de verdad original, permite desacoplar la ingesta inicial del procesamiento en tiempo real.

AWS Lambda (Función 1: Preprocesamiento):

Función: Se dispara mediante eventos de S3, extrae las líneas de texto del archivo `.csv` entrante, las parsea, valida los tipos de datos mínimos y las inyecta en el flujo de streaming.

Justificación: Evita mantener servidores EC2 encendidos las 24 horas del día. Su naturaleza elástica escala horizontalmente de forma automática en concordancia exacta con el volumen de archivos depositados en S3.

Amazon Kinesis Data Streams:

Función: Actúa como el *Backbone* de mensajería masiva y de baja latencia del sistema, reteniendo temporalmente los registros de tráfico ordenados de forma secuencial por llave de partición.

Justificación: Funciona como un búfer intermedio desacoplador (*absorber* de picos de tráfico), aislando el componente de ingesta rápida de los tiempos de respuesta del modelo de Machine Learning, garantizando que no se pierda ningún evento de red ante sobrecargas del sistema.

AWS Lambda (Función 2: Inferencia en Tiempo Real):

Función: Realiza un consumo periódico (*polling*) constante sobre los fragmentos (*shards*) de Kinesis, empaqueta los datos de tráfico en formato de matriz tabular e invoca vía HTTP POST el Endpoint de SageMaker.

Justificación: Integración nativa a nivel de infraestructura con Kinesis (*Event Source Mapping*), reduciendo la complejidad del código a escribir y minimizando los costos operativos de cómputo.

Amazon SageMaker (Training & Endpoints):

Función: Aloja el modelo predictivo supervisado (XGBoost) entrenado y lo expone a través de un endpoint HTTPS persistente de baja latencia para predicciones síncronas individuales o por pequeños lotes.

Justificación: Abstrae la gestión de la infraestructura subyacente de inferencia de modelos matemáticos complejos, proveyendo alta disponibilidad incorporada y aislamiento del entorno de ejecución de Machine Learning.

Amazon DynamoDB:

Función: Almacén persistente NoSQL de escritura ultrarrápida donde se registran exclusivamente los eventos que el modelo de Machine Learning cataloga con la etiqueta de anomalía (`is_anomaly = 1`).

Justificación: Su arquitectura interna distribuida garantiza tiempos de respuesta de escritura en el orden de **milisegundos de un solo dígito**, permitiendo que los equipos de seguridad (SOC) consulten incidentes en tiempo real sin cuellos de botella de bloqueo de tablas tradicionales.

Amazon CloudWatch:

Función: Centraliza la salida estándar (`stdout`) de las Lambdas, métricas operacionales de Kinesis y SageMaker, y gestiona las métricas personalizadas escritas en DynamoDB para disparar alarmas ante ataques masivos.

Justificación: Capa transversal indispensable en AWS que unifica la observabilidad y el gobierno operacional de soluciones distribuidas complejas sin requerir agentes de terceros.

4. Flujo de Datos y Tránsito Exhaustivo de una Petición

[S3 Event] → [Lambda Preproc] —(Payload JSON)—> [Kinesis] → [Lambda Inf

1. **Ingesta Original:** Una sonda de firewall del cliente finaliza la escritura de un archivo de tráfico y lo sube al bucket S3.
2. **Disparador Orientado a Eventos:** S3 detecta el fin de la escritura y emite una notificación JSON hacia el servicio interno de AWS Lambda.
3. **Filtrado Inicial y Streaming:** La función `fn-preprocess-logs` descarga el objeto en memoria (aprovechando flujos continuos o *streams* de lectura si el archivo supera el megabyte), formatea cada fila convirtiéndola en un registro JSON limpio y realiza una llamada por SDK a la API `PutRecords` de Amazon Kinesis.
4. **Buffer y Entrega Agrupada:** Amazon Kinesis recibe los registros, les asigna un número de secuencia inmutable según su orden de llegada y los retiene en sus discos distribuidos. El mapeo de origen de eventos de AWS Lambda lee estos mensajes en micro-lotes configurables (por ejemplo, cada 100 registros o cada 1 segundo).
5. **Evaluación Matemática:** La función `fn-rt-inference` toma el micro-lote, transforma los objetos JSON crudos en una matriz numérica optimizada de entradas tabulares sin cabeceras (requerimiento estricto del algoritmo XGBoost de SageMaker) e invoca la API `InvokeEndpoint` de SageMaker.
6. **Resolución de la Inferencia:** El contenedor de SageMaker recibe el vector de características, ejecuta el árbol de decisión del modelo matemático XGBoost en memoria y devuelve una respuesta síncrona en texto plano con la probabilidad de la anomalía (un valor flotante entre `0.0` y `1.0`).
7. **Persistencia del Alerta:** Si la predicción supera el umbral de discriminación establecido (por ejemplo, ≥ 0.5 , resultando en una predicción binaria de `1`), la

función Lambda formatea de inmediato una estructura JSON de persistencia y ejecuta un comando `PutItem` de DynamoDB con las IPs involucradas, métricas y la estampa de tiempo exacta.

5. Beneficios y Limitaciones Técnicas de la Arquitectura

Beneficios

Arquitectura Serverless y Autotrazo: No existen sistemas operativos que parchar, actualizar ni configurar a nivel de red básica, reduciendo drásticamente el costo de propiedad (*TCO*).

Desacoplamiento Estricto: Si el endpoint de SageMaker requiere ser actualizado, reentrenado o reiniciado, Kinesis puede retener el tráfico de logs en su buffer hasta por un periodo de **7 días por defecto** sin que se pierda un solo dato.

Escalabilidad Horizontal Transparente: La capacidad de procesamiento incrementa bajo demanda aumentando el número de *shards* en Kinesis y permitiendo concurrencia masiva controlada en las funciones Lambda.

Limitaciones Técnicas

Latencia de Inferencia Síncrona: A pesar de ser tiempo real, la invocación HTTP a través de Lambdas hacia un Endpoint de SageMaker introduce un tiempo de viaje redondo (*Round-Trip Time* o RTT) de entre **15ms a 50ms** por lote. No es apto para entornos industriales de ultra-baja latencia sub-milisegundo (como trading de alta frecuencia), pero es ideal para analítica de ciberseguridad corporativa.

Límites de Concurrencia de AWS Lambda: Si hay una tormenta de subidas masivas en S3, la Lambda de preprocesamiento podría colisionar temporalmente con el límite de concurrencia de la cuenta de AWS si no se parametriza de forma adecuada una concurrencia reservada.

Por favor, indícame si estás conforme con esta fundamentación arquitectónica para proceder de inmediato con la **Segunda Parte: Creación Paso a Paso de Todos los Recursos AWS**, asumiendo que partes desde una cuenta de AWS completamente en blanco desde la consola web.

Continúa

Asumiendo que estás frente a una **cuenta de AWS completamente vacía y nueva**, iniciaremos de inmediato la **Segunda Parte**. Vamos a crear toda la infraestructura base

de manera manual utilizando la Consola de Administración de AWS, paso a paso, pantalla por pantalla, para que ningún detalle quede al azar.

Segunda Parte: Creación Paso a Paso de Todos los Recursos AWS

1. Creación de Roles de Ejecución IAM (Identity and Access Management)

Antes de desplegar los servicios, necesitamos crear las identidades (*Roles*) que permitirán a nuestras funciones Lambda y a SageMaker comunicarse entre sí de forma segura.

Paso 1.1: Rol de Ejecución para las Funciones Lambda (`Lambda-Network-Analysis-Role`)

1. En la barra de búsqueda superior de la consola de AWS, escribe **IAM** y selecciona el primer resultado bajo servicios.
2. En el panel de navegación izquierdo, haz clic en **Roles**.
3. Haz clic en el botón naranja **Create role** (Crear rol).
4. **Select type of trusted entity (Seleccionar entidad de confianza):** Selecciona **AWS service** (Servicio de AWS).
5. **Service or use case (Servicio o caso de uso):** En el menú desplegable, selecciona **Lambda** y haz clic en el botón de radio junto a **Lambda**. Haz clic en **Next** (Siguiente).
6. **Add permissions (Añadir permisos):** En la barra de búsqueda de políticas, busca `AWSLambdaKinesisExecutionRole` y marca la casilla de verificación a su izquierda.
7. Limpia el buscador y ahora busca `AmazonDynamoDBFullAccess` y marca la casilla de verificación.
8. Limpia el buscador, busca `AmazonSageMakerFullAccess` y marca la casilla de verificación.
9. Limpia el buscador, busca `AmazonS3ReadOnlyAccess` y marca la casilla de verificación.
10. Haz clic en **Next** (Siguiente).
11. **Role name (Nombre del rol):** Escribe exactamente `Lambda-Network-Analysis-Role` .

12. **Description (Descripción):** Escribe Rol para que las Lambdas lean S3, Kinesis, invoquen SageMaker y escriban en DynamoDB .
13. Deja todo lo demás por defecto y haz clic en **Create role** (Crear rol) al final de la página.

Paso 1.2: Rol de Ejecución para Amazon SageMaker (`SageMaker-Execution-Role`)

1. En la misma pantalla de **IAM > Roles**, haz clic nuevamente en el botón naranja **Create role**.
2. **Select type of trusted entity:** Selecciona **AWS service**.
3. **Service or use case:** Busca y selecciona **SageMaker** en el menú desplegable, luego marca el botón de radio junto a **SageMaker - Execution** (o simplemente **SageMaker**). Haz clic en **Next**.
4. **Add permissions:** De forma predeterminada, AWS asociará la política `AmazonSageMakerFullAccess` . Deja esa casilla marcada.
5. En el buscador de políticas, busca `AmazonS3FullAccess` y marca la casilla (necesario para guardar los modelos e interactuar con el Data Lake de entrenamiento).
6. Haz clic en **Next**.
7. **Role name:** Escribe exactamente `SageMaker-Execution-Role` .
8. Deja todo lo demás por defecto y haz clic en **Create role** al final de la página.

2. Creación del Storage Layer: Amazon S3 (Simple Storage Service)

Crearemos el bucket que funcionará como el receptor de los logs crudos y repositorio de Machine Learning.

1. En la barra de búsqueda superior de la consola, escribe **S3** y haz clic en el servicio.
2. Haz clic en el botón naranja **Create bucket** (Crear bucket).
3. **Bucket name (Nombre del bucket):** Escribe un nombre globalmente único en minúsculas (por ejemplo: `network-logs-raw-[tu-nombre-o-id-unico]`). *A partir de aquí lo llamaremos `network-logs-raw-bucket` .*
4. **AWS Region (Región de AWS):** Selecciona la misma región donde vas a trabajar todo el proyecto (Recomendado: **US East (N. Virginia) us-east-1** o **US West (Oregon) us-west-2**).

5. **Object Ownership (Propiedad del objeto):** Deja seleccionada la opción por defecto **ACLs disabled (recommended)**.
6. **Block Public Access settings for this bucket (Bloquear acceso público):** Asegúrate de que la casilla **Block all public access** esté marcada (Queremos proteger nuestros datos de tráfico de red).
7. **Bucket Versioning (Versión del bucket):** Selecciona **Disable** (Deshabilitar) para simplificar y evitar costos adicionales en el entorno académico.
8. **Default encryption (Cifrado predeterminado):** Deja seleccionada la opción por defecto **Server-side encryption with Amazon S3 managed keys (SSE-S3)**.
9. Desplázate hasta el final de la página y haz clic en el botón naranja **Create bucket**.

3. Creación del Bus de Datos: Amazon Kinesis Data Streams

Crearemos el canal de streaming encargado de procesar el flujo continuo de registros.

1. En la barra de búsqueda superior de la consola, escribe **Kinesis** y selecciona el servicio.
2. En el panel principal, haz clic en el botón naranja **Create data stream** (Crear flujo de datos).
3. **Data stream name (Nombre del flujo de datos):** Escribe exactamente `stream-network-data` .
4. **Data stream capacity (Capacidad del flujo de datos):**

Selecciona la opción **Provisioned** (Aprovisionado). *(Nota: On-demand es excelente para producción, pero Provisioned con 1 Shard entra de forma óptima en los límites de pruebas académicas controladas).*

Number of shards (Número de fragmentos): Escribe `1` . (Esto proporciona 1 MB/s de entrada y 2 MB/s de salida, más que suficiente para procesar miles de registros de nuestro CSV).
5. Desplázate hasta el fondo y haz clic en el botón naranja **Create data stream**. Espera unos segundos a que el estado cambie de *Creating* a *Active*.

4. Creación del Almacenamiento NoSQL: Amazon DynamoDB

Crearemos la base de datos ultrarrápida donde guardaremos únicamente los registros catalogados como anomalías de red.

1. En la barra de búsqueda superior de la consola, escribe **DynamoDB** y selecciona el servicio.

2. En el panel central, haz clic en el botón naranja **Create table** (Crear tabla).
3. **Table name (Nombre de la tabla):** Escribe exactamente `network_anomalies` .
4. **Partition key (Clave de partición):** Escribe exactamente `source_ip` y en el menú desplegable de tipo de dato selecciona **String** (Cadena).
5. **Sort key (Clave de ordenación):** Marca la casilla de verificación adyacente y escribe exactamente `timestamp` . En el menú desplegable selecciona **String**.
6. **Table settings (Configuración de la tabla):** Selecciona el botón de radio de **Custom settings** (Configuración personalizada).
7. Desplázate hacia abajo hasta la sección **Table class (Clase de tabla)** y asegúrate de que esté marcado **DynamoDB Standard**.
8. En la sección **Read/write capacity settings (Configuración de capacidad):**
 - Selecciona **Provisioned** (Aprovisionado).
 - Desactiva la casilla de **Amazon DynamoDB auto scaling** tanto para lectura como para escritura para evitar recursos excedentes.
 - Read capacity units (RCU):** Escribe `5` .
 - Write capacity units (WCU):** Escribe `5` .
9. Deja el resto de las opciones por defecto (Cifrado propiedad de AWS, sin índices secundarios por ahora) y haz clic en el botón naranja **Create table**.

5. Configuración de Entornos de Cómputo: AWS Lambda

Procederemos a instanciar los cascarones de ejecución de nuestras dos funciones Lambda. El código funcional de negocio lo inyectaremos en los capítulos dedicados del backend.

Paso 5.1: Crear Lambda de Preprocesamiento (`fn-preprocess-logs`)

1. En la barra de búsqueda superior de la consola, escribe **Lambda** y selecciona el servicio.
2. Haz clic en el botón naranja **Create function** (Crear función).
3. Selecciona la opción por defecto **Author from scratch** (Crear desde cero).
4. **Function name (Nombre de la función):** Escribe exactamente `fn-preprocess-logs` .
5. **Runtime (Entorno de ejecución):** Abre el menú desplegable y selecciona **Python 3.12**.

6. **Architecture (Arquitectura):** Selecciona `x86_64`.
7. Despliega la sección **Change default execution role** (Cambiar rol de ejecución predeterminado):

 Selecciona **Use an existing role** (Usar un rol existente).

 En el menú desplegable, selecciona el rol que creamos en el paso 1.1:
 `Lambda-Network-Analysis-Role` .
8. Haz clic en el botón naranja **Create function**.

Paso 5.2: Crear Lambda de Inferencia en Tiempo Real (`fn-rt-inference`)

1. Regresa a la pantalla principal de **AWS Lambda** y haz clic nuevamente en **Create function**.
2. Selecciona **Author from scratch**.
3. **Function name:** Escribe exactamente `fn-rt-inference` .
4. **Runtime:** Selecciona **Python 3.12**.
5. **Architecture:** Selecciona `x86_64`.
6. Despliega la sección **Change default execution role:**

 Selecciona **Use an existing role**.

 Selecciona el rol: `Lambda-Network-Analysis-Role` .
7. Haz clic en el botón naranja **Create function**.

6. Configuración de Disparadores e Integraciones de Infraestructura

Ahora que los componentes estáticos existen, debemos cablear las uniones de eventos a nivel de AWS.

Integración A: S3 hacia `fn-preprocess-logs`

1. Estando en la pantalla de la función Lambda `fn-preprocess-logs` , ubica el diagrama visual superior y haz clic en el botón **+ Add trigger** (+ Añadir disparador).
2. En el menú desplegable **Select a source**, busca y selecciona **S3**.
3. **Bucket:** Selecciona el bucket que creaste en la sección 2 (`network-logs-raw-bucket`).
4. **Event types (Tipos de evento):** Deja seleccionada la opción por defecto **All object create events** (`s3:ObjectCreated:*`).

5. **Prefix (Prefijo):** Déjalo completamente vacío.
6. **Suffix (Sufijo):** Escribe `.csv`. (Esto garantiza que la Lambda solo despierte cuando se suban archivos de datos tabulares terminados en dicha extensión).
7. Desplázate al fondo, marca la casilla de advertencia de **Recursive invocation** (declarando que entiendes que tu Lambda no debe escribir en el mismo bucket para evitar bucles infinitos) y haz clic en el botón naranja **Add** (Añadir).

Integración B: Kinesis hacia `fn-rt-inference`

1. Ve a la pantalla de la función Lambda `fn-rt-inference` (puedes ir al menú de la izquierda en Lambda > Functions y hacer clic sobre ella).
2. En el diagrama visual superior, haz clic en el botón **+ Add trigger**.
3. En el menú desplegable **Select a source**, busca y selecciona **Kinesis**.
4. **Kinesis stream:** Selecciona el flujo creado en la sección 3 (`stream-network-data`).
5. **Batch size (Tamaño del lote):** Cambia el valor predeterminado a `100` (procesará un máximo de 100 registros de red en una sola invocación de inferencia).
6. **Starting position (Posición inicial):** Selecciona **Latest** (Último) para procesar los registros nuevos que entran al flujo a partir de la activación.
7. Deja el resto de las opciones por defecto y haz clic en el botón naranja **Add**.

Hemos completado la configuración y el aprovisionamiento de toda la topología de red y mensajería en AWS. Por favor, confirmame tu conformidad con este despliegue de infraestructura base para continuar con la **Tercera Parte: Bases de Datos** (donde definiremos las estructuras de datos nativas del proyecto).

Continúa

Procedemos de inmediato con la **Tercera Parte**, donde definiremos rigurosamente el esquema de datos nativo para nuestro motor de base de datos NoSQL No-Relacional (Amazon DynamoDB) y el diseño del almacenamiento analítico, de acuerdo con las especificaciones del caso de estudio. PDF

Tercera Parte: Bases de Datos y Estructuras de Almacenamiento

Al trabajar con una arquitectura serverless basada en **Amazon DynamoDB**, no se emplean scripts tradicionales de `CREATE DATABASE` o `CREATE TABLE` en dialecto SQL (relacional), debido a que es un motor NoSQL orientado a documentos clave-valor clave-ordenación.

Sin embargo, para cumplir con el rigor de un entorno de ingeniería y dar una guía completa e inmutable, presentaremos el esquema técnico de datos mediante la notación JSON nativa de la API de AWS (`DynamoDB AttributeDefinitions`), junto con el esquema del repositorio analítico de logs en S3.

1. Modelo de Datos en Amazon DynamoDB (`network_anomalies`)

La tabla almacenará **únicamente** los registros que el modelo de Machine Learning identifique con sospecha de ataque (`is_anomaly = 1`). PDF

Definición del Esquema Físico (Estructura de Claves)

Clave de Partición (Partition Key / Hash): `source_ip` (Tipo: `String`)

Justificación Técnica: Permite agrupar y consultar de manera eficiente todas las actividades sospechosas provenientes de una misma dirección IP de origen (potencial atacante) en un solo fragmento lógico.

Clave de Ordenación (Sort Key / Range): `timestamp` (Tipo: `String`)

Justificación Técnica: Permite organizar cronológicamente los eventos de una IP en específico, facilitando consultas de series de tiempo para análisis forense (ej. detectar escaneos rápidos u oleadas de fuerza bruta).

Script de Inicialización y Creación Nativo (CLI / SDK format)

Este JSON representa la estructura exacta de aprovisionamiento de la tabla bajo buenas prácticas:

JSON



```
{
  "TableName": "network_anomalies",
  "AttributeDefinitions": [
    {
      "AttributeName": "source_ip",
```



```

        "AttributeType": "S"
    },
    {
        "AttributeName": "timestamp",
        "AttributeType": "S"
    }
],
"KeySchema": [
    {
        "AttributeName": "source_ip",
        "KeyType": "HASH"
    },
    {
        "AttributeName": "timestamp",
        "KeyType": "RANGE"
    }
],
"ProvisionedThroughput": {
    "ReadCapacityUnits": 5,
    "WriteCapacityUnits": 5
}
}

```

Esquema de Documentos Extendidos (Atributos Dinámicos)

A diferencia de SQL, DynamoDB no requiere compilar columnas opcionales. Cada registro guardará la estructura del dataset proporcionado: [PDF](#)

JSON



```

{
  "source_ip": "5.6.7.8",
  "timestamp": "2025-07-18 10:00:06",
  "source_port": 12345,
  "destination_ip": "10.0.0.5",
  "destination_port": 22,
  "protocol": "TCP",
  "bytes_sent": 50,
  "bytes_received": 50,
  "duration": 0.5,
  "flag": "S0",
  "pkts_sent": 1,
  "pkts_received": 0,
  "service": "ssh",
  "num_failed_logins": 5,
}

```

```
"anomaly_score": 0.982,  
"is_anomaly": 1  
}
```

2. Estructura del Data Lake de Objetos en Amazon S3

Para el almacenamiento de datos históricos y logs crudos en el bucket S3 (`network-logs-raw-bucket`), se utilizará una estrategia de **Particionamiento por Clave de Tiempo**. Esto optimiza los costos de almacenamiento y facilita las tareas del componente *SageMaker Processing* al leer el histórico de entrenamiento. [PDF + 1](#)

La topología de carpetas lógicas (*prefixes*) dentro del bucket se estructurará de la siguiente manera:

```
network-logs-raw-bucket/  
├── raw-ingest/  
│   ├── year=2026/  
│   │   ├── month=07/  
│   │   │   ├── day=18/  
│   │   │   │   └── network_traffic_raw_1000.csv  
├── sagemaker-train-data/  
│   ├── train/  
│   │   └── network_traffic_dataset_train.csv  
│   └── validation/  
│       └── network_traffic_dataset_val.csv  
└── sagemaker-model-artifacts/  
    └── xgboost-anomalies-v1.tar.gz
```

`raw-ingest/` : Destino continuo de los logs crudos que activan la Lambda de preprocesamiento. [PDF](#)

`sagemaker-train-data/` : Datos limpios, balanceados y divididos, listos para que las instancias temporales de SageMaker ejecuten los algoritmos de entrenamiento. [PDF](#)

`sagemaker-model-artifacts/` : Directorio exclusivo donde SageMaker empaqueta y exporta el archivo binario del modelo entrenado (`model.tar.gz`). [PDF](#)

Por favor, indícame si estás conforme con esta definición de esquemas de almacenamiento para avanzar de inmediato a la **Cuarta Parte: Desarrollo** (Continúa donde construiremos todo el árbol de código fuente ejecutable para las funciones Lambda y la simulación del pipeline en Python).

Procedemos de inmediato con la **Cuarta Parte: Desarrollo del Backend**, donde generaremos el código completo, funcional y listo para ejecutarse de los componentes del sistema (funciones Lambda). PDF

Dado que nuestro entorno de ejecución es AWS Lambda Serverless sobre Python 3.12, implementaremos el patrón arquitectónico estándar para FaaS (*Function as a Service*), estructurado mediante capas de Controladores (*Event Handlers*), Servicios (*Business Logic*) e Integraciones (*Clients/Repositories*).

Cuarta Parte: Desarrollo del Backend (AWS Lambda Services)

1. Árbol de Directorios del Proyecto Backend

Crearemos la estructura de archivos local para empaquetar y desplegar ambas funciones.

Plaintext



```
network-security-backend/
|
├── fn_preprocess_logs/
|   ├── index.py           # Handler de AWS Lambda (Punto de entrada)
|   ├── s3_service.py      # Lógica para leer y procesar streams de S3
|   └── kinesis_client.py  # Integración con Kinesis Data Streams
|
├── fn_rt_inference/
|   ├── index.py           # Handler de AWS Lambda (Punto de entrada)
|   ├── sm_inference_client.py # Invocador del Endpoint de SageMaker
|   └── dynamodb_repository.py # Persistencia de alertas de anomalías
|
└── config.env             # Referencia de Variables de Entorno
```

2. Código de la Función 1: Preprocesamiento de Logs (fn_preprocess_logs)

Esta función es activada de manera síncrona/asíncrona por S3 cuando se deposita un .csv de red. Descarga el archivo línea por línea de manera eficiente y lo deposita en

Archivo: network-security-backend/fn_preprocess_logs/kinesis_client.py

Python



```
import json
import boto3
import os
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

class KinesisClient:
    def __init__(self):
        self.client = boto3.client('kinesis')
        self.stream_name = os.environ.get('KINESIS_STREAM_NAME', 'stream-netw

    def send_records(self, records):
        """
        Envía un lote de registros formateados a Amazon Kinesis Data Streams.
        """
        kinesis_records = []
        for record in records:
            # Usamos source_ip como PartitionKey para mantener consistencia d
            partition_key = record.get('source_ip', '0.0.0.0')
            kinesis_records.append({
                'Data': json.dumps(record).encode('utf-8'),
                'PartitionKey': partition_key
            })

        try:
            response = self.client.put_records(
                Records=kinesis_records,
                StreamName=self.stream_name
            )
            failed_count = response.get('FailedRecordCount', 0)
            if failed_count > 0:
                logger.error(f"Error: {failed_count} registros fallaron al in
            else:
                logger.info(f"Insertados exitosamente {len(kinesis_records)}
            return response
        except Exception as e:
```

```
logger.error(f"Fallo crítico enviando datos a Kinesis: {str(e)}")
raise e
```

Archivo: network-security-backend/fn_preprocess_logs/s3_service.py

Python



```
import csv
import codecs
import logging

logger = logging.getLogger()

class S3LogProcessorService:
    def __init__(self):
        pass

    def parse_csv_stream(self, s3_stream_body):
        """
        Lee el flujo de bytes de S3, lo procesa línea por línea como CSV
        y devuelve una lista de diccionarios limpios.
        """
        records = []
        # Wrapper para leer bytes como string texto plano UTF-8
        csv_reader = csv.DictReader(codecs.getreader('utf-8')(s3_stream_body))

        for row in csv_reader:
            try:
                # Sanitización básica de tipos de datos tabulares
                processed_row = {
                    "timestamp": row["timestamp"].strip(),
                    "source_ip": row["source_ip"].strip(),
                    "source_port": int(row["source_port"]),
                    "destination_ip": row["destination_ip"].strip(),
                    "destination_port": int(row["destination_port"]),
                    "protocol": row["protocol"].strip(),
                    "bytes_sent": int(row["bytes_sent"]),
                    "bytes_received": int(row["bytes_received"]),
                    "duration": float(row["duration"]),
                    "flag": row["flag"].strip(),
                    "pkts_sent": int(row["pkts_sent"]),
                    "pkts_received": int(row["pkts_received"]),
                    "service": row["service"].strip(),
                    "num_failed_logins": int(row["num_failed_logins"])
                }
```

```

        records.append(processed_row)
    except KeyError as ke:
        logger.warning(f"Fila omitida por ausencia de columna requeri
        continue
    except ValueError as ve:
        logger.warning(f"Fila omitida por error de conversión de tipo
        continue

return records

```

Archivo: network-security-backend/fn_preprocess_logs/index.py

Python



```

import boto3
import logging
from s3_service import S3LogProcessorService
from kinesis_client import KinesisClient

logger = logging.getLogger()
logger.setLevel(logging.INFO)

s3_client = boto3.client('s3')
processor_service = S3LogProcessorService()
kinesis_client = KinesisClient()

def handler(event, context):
    """
    Punto de entrada disparado por Amazon S3 (s3:ObjectCreated)
    """
    logger.info(f"Evento S3 recibido: {event}")

    try:
        # Extraer Bucket y Key del evento S3
        bucket = event['Records'][0]['s3']['bucket']['name']
        key = event['Records'][0]['s3']['object']['key']
        logger.info(f"Procesando archivo S3: s3://{bucket}/{key}")

        # Obtener el objeto desde S3
        s3_response = s3_client.get_object(Bucket=bucket, Key=key)

        # Parsear las líneas del CSV en memoria administrada
        parsed_records = processor_service.parse_csv_stream(s3_response['Body'])
        logger.info(f"Total de registros válidos a transmitir: {len(parsed_re

```

```

# Batching de envío a Kinesis (Kinesis soporta hasta 500 registros por
batch_size = 400
for i in range(0, len(parsed_records), batch_size):
    current_batch = parsed_records[i:i + batch_size]
    kinesis_client.send_records(current_batch)

return {
    "statusCode": 200,
    "body": f"Procesamiento exitoso del archivo {key}. Registros envi
}

except Exception as e:
    logger.error(f"Fallo general en la ejecución de la Lambda de preproce
raise e

```

3. Código de la Función 2: Inferencia en Tiempo Real (fn_rt_inference)

Esta función es despertada de forma continua por Kinesis, transforma el payload JSON a formato plano exigido por SageMaker (CSV sin cabecera), evalúa y persiste anomalías si `is_anomaly == 1`. PDF

Archivo: network-security-backend/fn_rt_inference/sm_inference_client.py

Python



```

import boto3
import os
import logging

logger = logging.getLogger()

class SageMakerInferenceClient:
    def __init__(self):
        self.client = boto3.client('sagemaker-runtime')
        self.endpoint_name = os.environ.get('SAGEMAKER_ENDPOINT_NAME', 'ep-ne

    def invoke_model(self, csv_line_payload):
        """
        Invoca síncronamente el endpoint de SageMaker enviando datos en forma
        """
        try:
            response = self.client.invoke_endpoint(
                EndpointName=self.endpoint_name,

```

```

        ContentType='text/csv',
        Body=csv_line_payload
    )
    # Leer el score devuelto por el contenedor XGBoost
    result = response['Body'].read().decode('utf-8').strip()
    return float(result)
except Exception as e:
    logger.error(f"Error invocando el endpoint de SageMaker: {str(e)}")
    # Devolvemos un score neutral seguro en caso de fallo para evitar
    return 0.0

```

Archivo: network-security-

backend/fn_rt_inference/dynamodb_repository.py

Python



```

import boto3
import os
import logging
from decimal import Decimal

logger = logging.getLogger()

class DynamoDBAnomalyRepository:
    def __init__(self):
        self.dynamodb = boto3.resource('dynamodb')
        self.table_name = os.environ.get('DYNAMODB_TABLE_NAME', 'network_anom')
        self.table = self.dynamodb.Table(self.table_name)

    def save_anomaly(self, data, anomaly_score):
        """
        Guarda el registro de red catalogado como anomalía en DynamoDB.
        Convertimos flotantes a Decimal (Requisito estricto de boto3 para Dyn
        """
        try:
            item = {
                'source_ip': data['source_ip'],
                'timestamp': data['timestamp'],
                'source_port': int(data['source_port']),
                'destination_ip': data['destination_ip'],
                'destination_port': int(data['destination_port']),
                'protocol': data['protocol'],
                'bytes_sent': int(data['bytes_sent']),
                'bytes_received': int(data['bytes_received']),
                'duration': Decimal(str(data['duration'])),
            }

```



```

        'flag': data['flag'],
        'pkts_sent': int(data['pkts_sent']),
        'pkts_received': int(data['pkts_received']),
        'service': data['service'],
        'num_failed_logins': int(data['num_failed_logins']),
        'anomaly_score': Decimal(str(round(anomaly_score, 4))),
        'is_anomaly': 1
    }
    self.table.put_item(Item=item)
    logger.info(f"ALERTA CRÍTICA: Anomalía registrada desde la IP {da
except Exception as e:
    logger.error(f"Error al guardar registro de anomalía en DynamoDB:

```

Archivo: network-security-backend/fn_rt_inference/index.py

Python



```

import base64
import json
import logging
from sm_inference_client import SageMakerInferenceClient
from dynamodb_repository import DynamoDBAnomalyRepository

logger = logging.getLogger()
logger.setLevel(logging.INFO)

sm_client = SageMakerInferenceClient()
db_repository = DynamoDBAnomalyRepository()

# Lista estricta del orden de características que espera el modelo entrenado
# Nota: Excluye 'timestamp', 'source_ip', 'destination_ip' y la columna objet
# Mapeo categórico interno simplificado para el modelo matemático (protocol,
def transform_to_features_vector(record):
    # Diccionarios de mapeo estático para simular la codificación del modelo
    protocol_map = {'TCP': 0, 'UDP': 1, 'ICMP': 2}
    flag_map = {'SF': 0, 'S0': 1, 'REJ': 2, 'RST': 3}
    service_map = {'http': 0, 'https': 1, 'dns': 2, 'ssh': 3, 'ftp': 4, 'smtp'

    features = [
        str(record['source_port']),
        str(record['destination_port']),
        str(protocol_map.get(record['protocol'], 0)),
        str(record['bytes_sent']),
        str(record['bytes_received']),
        str(record['duration']),

```

```

        str(flag_map.get(record['flag'], 0)),
        str(record['pkts_sent']),
        str(record['pkts_received']),
        str(service_map.get(record['service'], 0)),
        str(record['num_failed_logins'])
    ]
    return ",".join(features)

def handler(event, context):
    """
    Punto de entrada disparado por Amazon Kinesis Data Streams (Event Source
    """
    logger.info(f"Recibido lote de Kinesis con {len(event['Records'])} regist

    for kinesis_record in event['Records']:
        try:
            # Decodificar el payload en Base64 nativo de Kinesis
            payload_bytes = base64.b64decode(kinesis_record['kinesis']['data']
            record = json.loads(payload_bytes.decode('utf-8'))

            # Formatear línea CSV para enviar a SageMaker
            csv_payload = transform_to_features_vector(record)

            # Invocar modelo
            score = sm_client.invoke_model(csv_payload)

            # Si la probabilidad es mayor o igual a 0.5, se considera una ano
            if score >= 0.5:
                db_repository.save_anomaly(record, score)

        except Exception as e:
            logger.error(f"Error procesando registro individual del Stream: {
            continue # Garantizamos que un registro corrupto no detenga el pr

    return {"statusCode": 200, "body": "Lote procesado correctamente."}

```

4. Variables de Entorno de Configuración (config.env)

Estas llaves deben configurarse en el panel web de configuración de cada función Lambda en la consola de AWS:

Bash



```

# Para la función fn-preprocess-logs:
KINESIS_STREAM_NAME=stream-network-data

```

```
# Para la función fn-rt-inference:
SAGEMAKER_ENDPOINT_NAME=ep-network-anomaly
DYNAMODB_TABLE_NAME=network_anomalies
```

Por favor, indícame tu conformidad con el desarrollo de la lógica del backend para avanzar a la **Quinta Parte: Frontend**, donde construiremos un script interactivo o consola limpia que simule la carga de datos de red en tiempo real.

Continúa

Procedemos con la **Quinta Parte: Frontend**, donde crearemos una interfaz de control limpia y funcional basada en una consola de analítica web e interactiva. Como el documento no exige una interfaz gráfica web compleja y se enfoca en el comportamiento serverless, un panel de operaciones interactivo en Python es la mejor decisión técnica para simular la inyección de logs, ver el estado del flujo y capturar las alertas en tiempo real. [PDF + 1](#)

Quinta Parte: Panel de Operaciones e Ingesta (Frontend de Control)

Este componente local actuará como nuestra consola de operaciones corporativa. Permite dos acciones fundamentales:

1. **Simular Tráfico de Red:** Genera lotes de registros utilizando la lógica de `faker.ipynb` y los sube automáticamente a S3 para activar todo el pipeline de cómputo de la nube. [PDF + 1](#)
2. **Visualizar Alertas de Anomalías:** Realiza consultas directas a DynamoDB para mostrar los vectores de ataque capturados por el modelo en tiempo real. [PDF](#)

1. Código del Panel Interactivo (`dashboard.py`)

Archivo: `network-security-backend/dashboard.py`

Python



```
import os
import time
import boto3
import pandas as pd
from datetime import datetime
```

```

# Configuración de Clientes de AWS (Lee las credenciales locales de AWS CLI)
s3_client = boto3.client('s3')
dynamodb_resource = boto3.resource('dynamodb')

# Configuración de Variables (Modificar con tus recursos creados)
BUCKET_NAME = "network-logs-raw-bucket-remoto" # Reemplazar por tu nombre rea
TABLE_NAME = "network_anomalies"

def limpiar_pantalla():
    os.system('cls' if os.name == 'nt' else 'clear')

def mostrar_menu():
    print("=====
    print("          DASHBOARD DE OPERACIONES - DETECCIÓN DE ANOMALÍAS EN RED
    print("=====
    print(f" Fecha Actual: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print("-----
    print(" [1] Generar y Subir Lote de Logs Crudos a Amazon S3")
    print(" [2] Monitorear Alertas de Anomalías Recientes (DynamoDB)")
    print(" [3] Salir de la Consola")
    print("=====

def simular_y_subir_s3():
    limpiar_pantalla()
    print("[+] Cargando base de datos sintética de tráfico de red...")

# En un entorno real, aquí leeríamos del archivo generado por tu script d
# Simulamos un dataframe rápido basado en el formato exacto del examen pa
timestamp_str = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

# Inyectamos intencionalmente un tráfico normal y un ataque de Fuerza Bru
datos_ejemplo = [
    {
        "timestamp": timestamp_str, "source_ip": "192.168.1.50", "source_
        "destination_ip": "10.0.0.5", "destination_port": 443, "protocol"
        "bytes_sent": 4500, "bytes_received": 12000, "duration": 4.2, "fl
        "pkts_sent": 4, "pkts_received": 8, "service": "https", "num_fail
    },
    {
        "timestamp": timestamp_str, "source_ip": "5.6.7.8", "source_port"
        "destination_ip": "10.0.0.5", "destination_port": 22, "protocol":
        "bytes_sent": 50, "bytes_received": 50, "duration": 0.5, "flag":
        "pkts_sent": 1, "pkts_received": 0, "service": "ssh", "num_failed
    }
]

df = pd.DataFrame(datos_ejemplo)
filename = f"network_traffic_live_{int(time.time())}.csv"

```

```

df.to_csv(filename, index=False)

# Armar ruta lógica de Data Lake (Año/Mes/Día)
now = datetime.now()
s3_key = f"raw-ingest/year={now.year}/month={now.strftime('%m')}/day={now.strftime('%d')}"

print(f"[^] Subiendo archivo local '{filename}' a S3 en s3://{BUCKET_NAME}")
try:
    s3_client.upload_file(filename, BUCKET_NAME, s3_key)
    print("\n[✓] ¡Archivo enviado con éxito! El pipeline serverless ha sido ejecutado correctamente.")
except Exception as e:
    print(f"\n[X] Error subiendo archivo a AWS S3: {str(e)}")

if os.path.exists(filename):
    os.remove(filename)

input("\nPresiona Enter para regresar al menú principal...")

def consultar_alertas_dynamodb():
    limpiar_pantalla()
    print("=====")
    print("                        ALERTAS DE INCIDENTES ACTIVOS EN RED")
    print("=====")

    try:
        table = dynamodb_resource.Table(TABLE_NAME)
        # Realizamos un Scan limitado para auditoría en el Dashboard corporativo
        response = table.scan(Limit=10)
        items = response.get('Items', [])

        if not items:
            print(" [i] No se registran incidentes anómalos en la red actualmente.")
        else:
            for item in items:
                print(f" CRITICAL CRIME ALERT: Anomalía Detectada desde IP: {item['source_ip']}")
                print(f" └ Hora del Evento: {item['timestamp']}")
                print(f" └ Destino: {item['destination_ip']}:{item['destination_port']}")
                print(f" └ Score de Certeza del Modelo de ML: {float(item['ml_score'])}")
                print(f" └ Intentos Fallidos de Login: {int(item['num_failed_login'])}")
                print("-" * 69)

    except Exception as e:
        print(f"[X] Error al leer la base de datos central de alertas: {str(e)}")

    input("\nPresiona Enter para regresar al menú principal...")

def ejecutar_dashboard():
    while True:
        limpiar_pantalla()

```

```

mostrar_menu()
opcion = input("Selecciona una opción (1-3): ").strip()

if opcion == "1":
    simular_y_subir_s3()
elif opcion == "2":
    consultar_alertas_dynamodb()
elif opcion == "3":
    print("\nCerrando Consola de Seguridad. ¡Buen día!")
    break
else:
    print("\nOpción inválida. Intenta de nuevo.")
    time.sleep(1.5)

if __name__ == "__main__":
    ejecutar_dashboard()

```

Por favor, confírmame si deseas avanzar a la **Sexta Parte: Desarrollo de cada Servicio AWS**, donde nos enfocaremos en escribir y desplegar los scripts y especificaciones completas para entrenar y levantar el Endpoint de Machine Learning en Amazon SageMaker. [PDF](#)

Continúa

Procedemos con la **Sexta Parte: Desarrollo de cada Servicio AWS**, enfocándonos en el núcleo de Inteligencia Artificial del sistema: **Amazon SageMaker**. [PDF](#)

Aprenderemos cómo configurar el entorno, preparar el dataset de red basado en el script provisto, entrenar un modelo clasificador binario utilizando el algoritmo integrado **XGBoost** de SageMaker, y desplegarlo en un endpoint persistente en tiempo real. [PDF + 1](#)

Sexta Parte: Entrenamiento y Despliegue en Amazon SageMaker

1. El Script de Entrenamiento y Orquestación (`train_deploy.py`)

Para que un Ingeniero DevOps o Arquitecto pueda automatizar el proceso de Machine Learning sin depender exclusivamente de una interfaz visual de Jupyter Notebook,

utilizaremos el **AWS SDK para Python (Boto3)** y la librería especializada **SageMaker Python SDK**.

Este script realiza todo el ciclo de vida: lee los datos de entrenamiento, levanta una instancia efímera de cómputo para compilar las matrices, genera el artefacto matemático y lo publica en producción. PDF

Archivo: network-security-backend/train_deploy.py

Python



```
import os
import boto3
import sagemaker
from sagemaker.inputs import TrainingInput
from sagemaker.image_uris import retrieve

def iniciar_pipeline_ml():
    # 1. Inicializar la sesión de SageMaker y clientes de AWS
    session = sagemaker.Session()
    region = session.boto_region_name

    # Configurar los nombres exactos de tus recursos creados en los pasos pre
    bucket_name = "network-logs-raw-bucket-remoto" # Reemplazar con tu bucket
    role_arn = "arn:aws:iam::123456789012:role/SageMaker-Execution-Role" # Re

    prefix_data = "sagemaker-train-data"
    prefix_models = "sagemaker-model-artifacts"

    print(f"[+] Iniciando Pipeline de ML en la región: {region}")

    # 2. Definir las rutas del Data Lake en S3 para los canales de entrenamie
    s3_train_path = f"s3://{bucket_name}/{prefix_data}/train/network_traffic_
    s3_val_path = f"s3://{bucket_name}/{prefix_data}/validation/network_traff
    s3_output_path = f"s3://{bucket_name}/{prefix_models}/"

    # 3. Obtener la URI de la imagen de Docker oficial de XGBoost administrad
    # Usamos la versión estable 1.5-1
    container_uri = retrieve/framework="xgboost", region=region, version="1.5
    print(f"[^] Utilizando contenedor Docker de AWS XGBoost: {container_uri}"

    # 4. Configurar el Estimador (Estimator) de SageMaker
    # Define la infraestructura física temporal para el entrenamiento matemát
    xgb_estimator = sagemaker.estimator.Estimator(
        image_uri=container_uri,
        role=role_arn,
```

```

        instance_count=1,
        instance_type="ml.m5.xlarge", # Instancia estándar de cómputo general
        output_path=s3_output_path,
        sagemaker_session=session,
        use_spot_instances=True,      # Buena práctica: reduce costos usando
        max_run=3600,
        max_wait=3600
    )

# 5. Configurar los Hiperparámetros del Algoritmo XGBoost
# Configuración optimizada para Clasificación Binaria y datasets altamente
xgb_estimator.set_hyperparameters(
    objective="binary:logistic", # Devuelve una probabilidad entre 0.0 y
    num_round=100,               # Cantidad de árboles de decisión a con
    max_depth=5,                 # Profundidad máxima por árbol
    eta=0.2,                     # Tasa de aprendizaje (learning rate)
    scale_pos_weight=99          # Ajuste crítico de desequilibrio: ratio
)

# 6. Mapear los canales de entrada obligatorios en formato CSV
train_input = TrainingInput(s3_data=f"s3://{bucket_name}/{prefix_data}/tr
val_input = TrainingInput(s3_data=f"s3://{bucket_name}/{prefix_data}/vali

# 7. Ejecutar el Entrenamiento (Bloquea la ejecución hasta que la instanc
print("[+] Ejecutando comando FIT... Levantando infraestructura efímera e
xgb_estimator.fit({"train": train_input, "validation": val_input})
print("[✓] Entrenamiento completado con éxito. Artefacto exportado a S3.'

# 8. Desplegar el modelo a un Endpoint HTTP en tiempo real persistente
print("[+] Publicando el Endpoint de Producción ('ep-network-anomaly')...
xgb_predictor = xgb_estimator.deploy(
    initial_instance_count=1,
    instance_type="ml.t2.medium", # Instancia económica ideal para entorn
    endpoint_name="ep-network-anomaly"
)
print("[✓] ¡Endpoint operativo y escuchando peticiones en vivo de AWS Lan

if __name__ == "__main__":
    iniciar_pipeline_ml()

```

2. Preparación y Formateo de Datos Tabulares para XGBoost

Regla de Oro de SageMaker XGBoost: Para que el algoritmo nativo integrado pueda procesar correctamente archivos CSV durante el entrenamiento, se deben cumplir tres condiciones estrictas:

1. El archivo **no debe contener cabeceras** (remover nombres de columnas). PDF

2. La columna objetivo (`target`) debe ser la **primera columna** del archivo (columna 0). PDF
3. Todas las variables deben ser de tipo numérico (codificar variables de texto a enteros). PDF

A continuación se detalla cómo adaptar el script `faker.ipynb` provisto para guardar la salida en el formato exacto requerido antes de invocar el pipeline de entrenamiento:

IPYNB

Python



```
# Bloque de código a ejecutar localmente para preparar el dataset analítico:
import pandas as pd
from sklearn.model_selection import train_test_split

# Supongamos que ya generaste el 'network_traffic_dataset.csv' usando tu note
df_original = pd.read_csv('network_traffic_dataset.csv')

# 1. Reordenar: Mover 'is_anomaly' para que sea la primera columna
columnas = ['is_anomaly'] + [col for col in df_original.columns if col != 'is_anomaly']
df_reordenado = df_original[columnas]

# 2. Remover variables identificadoras no predictivas para el cálculo del modelo
# Eliminamos timestamp, source_ip y destination_ip porque causan sobreajuste
df_reordenado = df_reordenado.drop(columns=['timestamp', 'source_ip', 'destination_ip'])

# 3. Codificar variables categóricas (Label Encoding manual para consistencia)
protocol_map = {'TCP': 0, 'UDP': 1, 'ICMP': 2}
flag_map = {'SF': 0, 'S0': 1, 'REJ': 2, 'RST': 3}
service_map = {'http': 0, 'https': 1, 'dns': 2, 'ssh': 3, 'ftp': 4, 'smtp': 5}

df_reordenado['protocol'] = df_reordenado['protocol'].map(protocol_map).fillna(0)
df_reordenado['flag'] = df_reordenado['flag'].map(flag_map).fillna(0).astype(int)
df_reordenado['service'] = df_reordenado['service'].map(service_map).fillna(0)

# 4. Dividir el conjunto en un 80% para entrenamiento y un 20% para validación
train_df, val_df = train_test_split(df_reordenado, test_size=0.2, random_state=42)

# 5. Guardar a archivos CSV locales SIN cabeceras ni índices
train_df.to_csv('train_processed.csv', header=False, index=False)
val_df.to_csv('val_processed.csv', header=False, index=False)

print("[✓] Archivos locales preparados con éxito en formato estructurado para entrenamiento")
```

Una vez que estos archivos procesados (`train_processed.csv` y `val_processed.csv`) sean subidos mediante el AWS CLI u otra herramienta a los directorios del bucket S3 correspondientes (`sagemaker-train-data/train/` y `sagemaker-train-data/validation/`), el script `train_deploy.py` podrá ejecutarse sin inconvenientes.

Por favor, infórmame si estás de acuerdo con la estructuración del servicio analítico de Machine Learning para pasar de inmediato a la **Séptima Parte: Despliegue**, donde cubriremos los comandos e instalación completa de herramientas desde una terminal local para empaquetar las funciones Lambda y sincronizar el proyecto en la nube. PDF

Continúa

Procedemos de inmediato con la **Séptima Parte: Despliegue**, donde explicaremos paso a paso la instalación de las herramientas locales necesarias, el empaquetado zip de nuestras funciones Lambda y la inyección del código en AWS. PDF

Séptima Parte: Configuración del Entorno Local y Despliegue del Sistema

1. Instalación de Herramientas Locales Necesarias

Para interactuar con nuestra cuenta vacía de AWS desde la computadora, prepararemos la estación de trabajo instalando los siguientes componentes básicos:

A. AWS CLI (Command Line Interface)

Windows: Descarga y ejecuta el instalador MSI oficial desde

[<https://awscli.amazonaws.com/AWSCLIV2.msi>]

(<https://awscli.amazonaws.com/AWSCLIV2.msi>) .

Linux (Ubuntu/Debian): Ejecuta en tu terminal:

Bash



```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Configuración Inicial: Abre una terminal y escribe `aws configure` . Introduce tus credenciales de la cuenta AWS (`AWS Access Key ID` y `AWS Secret Access Key`), define tu región por defecto (ej. `us-east-1`) y el formato de salida (`json`).

B. Python 3.12 y Librerías de Datos

Asegúrate de tener instalado Python 3.12. Instala los paquetes requeridos globales ejecutando:

Bash



```
pip install boto3 pandas sagemaker scikit-learn faker
```

2. Empaquetado y Despliegue de las Funciones Lambda via AWS CLI

Como nuestras funciones Lambda utilizan módulos nativos y librerías básicas estándar incorporadas en el entorno oficial de Python de AWS (`boto3` , `json` , `csv` , `base64`), no requerimos compilar paquetes pesados dentro de un contenedor Docker local. Podemos comprimirlas directamente en formato `.zip` y subirlas mediante comandos directos. [PDF](#)

Paso A: Desplegar `fn-preprocess-logs`

1. Entra a la carpeta del proyecto en tu terminal local:

Bash



```
cd network-security-backend/fn_preprocess_logs/
```

2. Comprime el código del módulo de preprocesamiento:

Bash



```
# En Linux / Mac:  
zip -r ../fn_preprocess_logs.zip .
```

En Windows (PowerShell):

```
Compress-Archive -Path * -DestinationPath ../fn_preprocess_logs.zip -Force
```

3. Sube el paquete de código directamente a la infraestructura de AWS Lambda:

Bash



```
aws lambda update-function-code \  
  --function-name fn-preprocess-logs \  
  --zip-file fileb://../fn_preprocess_logs.zip
```

4. Configura la variable de entorno obligatoria para el streaming:

Bash



```
aws lambda update-function-configuration \  
  --function-name fn-preprocess-logs \  
  --environment "Variables={KINESIS_STREAM_NAME=stream-network-data}"
```

Paso B: Desplegar `fn-rt-inference`

1. Cambia al directorio de la segunda Lambda:

Bash



```
cd ../fn_rt_inference/
```

2. Genera el empaquetado del módulo de inferencia en tiempo real:

Bash



En Linux / Mac:

```
zip -r ../fn_rt_inference.zip .
```

En Windows (PowerShell):

```
Compress-Archive -Path * -DestinationPath ../fn_rt_inference.zip -Force
```

3. Sube el código binario a la nube:

Bash



```
aws lambda update-function-code \  
  --function-name fn-rt-inference \  
  --zip-file fileb://../fn_rt_inference.zip
```

4. Aplica las configuraciones del entorno remoto de ejecución:

Bash



```
aws lambda update-function-configuration \  
  --function-name fn-rt-inference \  
  --environment "Variables={SAGEMAKER_ENDPOINT_NAME=ep-network-anomaly
```

3. Verificación de Infraestructura Conectada

Para comprobar que los cables de nuestra arquitectura EDA (*Event-Driven Architecture*) quedaron perfectamente configurados de extremo a extremo, ejecutaremos los siguientes dos comandos de diagnóstico en la consola local:

Comprobar disparador S3:

Bash



```
aws lambda get-policy --function-name fn-preprocess-logs
```

Debe retornar un documento JSON de política que contenga el bloque `s3.amazonaws.com` indicando la suscripción a eventos de tu bucket.

Comprobar mapeo de Kinesis Stream:

Bash



```
aws lambda list-event-source-mappings --function-name fn-rt-inference
```

Debe mostrar un registro con el estado `Enabled` enlazando el stream de Kinesis con el consumidor de inferencia.

Por favor, infórmame si estás conforme con este proceso de empaquetado
despliegue para avanzar directamente a la **Octava Parte: Pruebas**, donde Continúa
inyecciones reales de datos para validar el comportamiento reactivo del sistema.

Procedemos de inmediato con la **Octava Parte: Pruebas**, donde implementaremos y validaremos de manera exhaustiva el funcionamiento de todo nuestro pipeline distribuido. Simularemos tanto un escenario exitoso (tráfico de red benigno) como un escenario de ataque (tráfico anómalo) para verificar la correcta integración de todos los componentes de AWS. PDF + 1

Octava Parte: Plan de Pruebas y Validación del Sistema

1. Casos de Prueba Estructurados

Para verificar la solución de extremo a extremo, evaluaremos dos registros específicos extraídos directamente del comportamiento esperado de nuestro dataset: PDF + 1

Caso de Prueba 1: Tráfico de Red Benigno (Flujo Normal)

Entrada (JSON):

JSON  

```
{
  "timestamp": "2026-07-18 10:00:01",
  "source_ip": "192.168.1.10",
  "source_port": 54321,
  "destination_ip": "10.0.0.5",
  "destination_port": 80,
  "protocol": "TCP",
  "bytes_sent": 1500,
  "bytes_received": 2000,
  "duration": 1.2,
  "flag": "SF",
  "pkts_sent": 16,
  "pkts_received": 12,
  "service": "http",
  "num_failed_logins": 0
}
```

Resultado Esperado: La función Lambda de preprocesamiento lo envía a Kinesis. La Lambda de inferencia lo extrae y lo evalúa con el Endpoint de SageMaker. El

modelo retorna un score bajo (≤ 0.5). El registro **NO** se almacena en DynamoDB (la red se considera segura). PDF + 3

Caso de Prueba 2: Tráfico de Red Anómalo (Ataque de Fuerza Bruta SSH)

Entrada (JSON):

JSON



```
{
  "timestamp": "2026-07-18 10:00:06",
  "source_ip": "5.6.7.8",
  "source_port": 12345,
  "destination_ip": "10.0.0.5",
  "destination_port": 22,
  "protocol": "TCP",
  "bytes_sent": 50,
  "bytes_received": 50,
  "duration": 0.5,
  "flag": "S0",
  "pkts_sent": 1,
  "pkts_received": 0,
  "service": "ssh",
  "num_failed_logins": 5
}
```

Resultado Esperado: Al procesarse por la cadena de eventos, el Endpoint de SageMaker XGBoost calcula un score alto (≥ 0.5) debido a los múltiples intentos fallidos de login y banderas anómalas. La Lambda de inferencia detecta la condición crítica e inserta inmediatamente una alerta persistente en la tabla de DynamoDB `network_anomalies`. PDF + 2

2. Archivo de Pruebas Automatizado (`test_pipeline.py`)

Crearemos un script local para automatizar el empaquetado del archivo CSV y la inyección síncrona en el bucket S3 para validar la reactividad completa de la nube.

PDF

Archivo: `network-security-backend/test_pipeline.py`

Python



```

import os
import time
import boto3
import pandas as pd
from datetime import datetime

s3_client = boto3.client('s3')
dynamodb_resource = boto3.resource('dynamodb')

BUCKET_NAME = "network-logs-raw-bucket-remoto" # Reemplazar con tu bucket re
TABLE_NAME = "network_anomalies"

def ejecutar_suite_pruebas():
    print("[+] Inicializando suite de pruebas funcionales para la infraestruc

    # 1. Crear el set de datos simulando ambos casos de prueba
    timestamp_actual = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    datos_prueba = [
        # Registro 1: Benigno
        {
            "timestamp": timestamp_actual, "source_ip": "192.168.1.10", "sour
            "destination_ip": "10.0.0.5", "destination_port": 80, "protocol":
            "bytes_sent": 1500, "bytes_received": 2000, "duration": 1.2, "fla
            "pkts_sent": 16, "pkts_received": 12, "service": "http", "num_fai
        },
        # Registro 2: Anómalo (Fuerza Bruta)
        {
            "timestamp": timestamp_actual, "source_ip": "5.6.7.8", "source_po
            "destination_ip": "10.0.0.5", "destination_port": 22, "protocol":
            "bytes_sent": 50, "bytes_received": 50, "duration": 0.5, "flag":
            "pkts_sent": 1, "pkts_received": 0, "service": "ssh", "num_failed
        }
    ]

    df = pd.DataFrame(datos_prueba)
    local_filename = "integration_test_traffic.csv"
    df.to_csv(local_filename, index=False)

    # 2. Subir el archivo a S3 para despertar el trigger EDA
    now = datetime.now()
    s3_key = f"raw-ingest/year={now.year}/month={now.strftime('%m')}/day={now

    print(f"[^] Inyectando archivo de prueba en S3: s3://{BUCKET_NAME}/{s3_ke
    s3_client.upload_file(local_filename, BUCKET_NAME, s3_key)

    print("[i] Esperando 10 segundos para que el pipeline asíncrono procese e
    time.sleep(10)

```



```

# 3. Validar resultados leyendo DynamoDB
print("[+] Consultando base de datos de alertas para validar persistencia")
tabla = dynamodb_resource.Table(TABLE_NAME)

try:
    # Buscamos específicamente la IP sospechosa del caso de prueba 2
    respuesta = tabla.get_item(
        Key={
            'source_ip': '5.6.7.8',
            'timestamp': timestamp_actual
        }
    )

    item = respuesta.get('Item')
    if item:
        print("\n=====
        print("  [✓] ¡PRUEBA EXITOSA! El pipeline capturó y registró la a
        print("=====
        print(f" Atacante Identificado: {item['source_ip']}")
        print(f" Severidad (Score ML): {float(item['anomaly_score']) * 10
        print(f" Servicio Vulnerado: {item['service']}")
        print("=====
    else:
        print("\n[✗] Error de Integración: La anomalía no fue detectada o

except Exception as e:
    print(f"[✗] Error ejecutando la validación en DynamoDB: {str(e)}")

# Limpieza
if os.path.exists(local_filename):
    os.remove(local_filename)

if __name__ == "__main__":
    ejecutar_suite_pruebas()

```

Por favor, confírmame si estás de acuerdo con esta estrategia de pruebas de integración para pasar de inmediato a la **Novena Parte: Monitoreo**, donde aprenderemos a auditar las salidas de logs y trazas mediante Amazon CloudWatch.

PDF

Continúa

Procedemos con la **Novena Parte: Monitoreo, Auditoría y Mantenimiento**, centrada en asegurar el control operacional, la observabilidad y el ciclo de vida continuo de nuestra arquitectura serverless mediante **Amazon CloudWatch**. [PDF](#)

Novena Parte: Monitoreo y Observabilidad (Amazon CloudWatch)

En sistemas basados en arquitecturas dirigidas por eventos (EDA), la observabilidad es vital para rastrear fallos silenciosos (como excepciones al invocar el endpoint de SageMaker o registros corruptos en el Stream). Utilizaremos CloudWatch para centralizar logs, definir alarmas métricas y auditar la salud del pipeline. [PDF + 1](#)

1. Extracción de Logs de Error mediante CloudWatch Logs Insights

Cuando una función Lambda arroja un error a través de `logger.error()`, AWS almacena automáticamente la traza de ejecución en un grupo de registros (`/aws/lambda/fn-rt-inference`).

Para auditar y extraer rápidamente los fallos de inferencia sin revisar millones de líneas manualmente, podemos ejecutar la siguiente consulta **CloudWatch Insights** en la consola de AWS:

SQL



```
fields @timestamp, @message, @logStream
| filter @message like /Error/ or @message like /Fallo/
| sort @timestamp desc
| limit 50
```

¿Qué hace esta consulta? Filtra en tiempo real el flujo secuencial de logs buscando excepciones críticas de conectividad con SageMaker o DynamoDB, devolviendo los 50 incidentes más recientes ordenados cronológicamente. [PDF](#)

2. Creación Automática de Alarmas Métricas vía AWS CLI

Para garantizar que el equipo de DevSecOps reciba notificaciones inmediatas si el pipeline empieza a fallar catastróficamente, configuraremos una alarma métrica de CloudWatch. Esta alarma monitoreará si la función Lambda de inferencia en tiempo real supera un umbral crítico de errores.

Ejecuta el siguiente comando en tu terminal para aprovisionar la alerta:

```
aws cloudwatch put-metric-alarm \  
  --alarm-name "Alarma-Fallos-Inferencia-Red" \  
  --alarm-description "Se dispara si la Lambda de inferencia en tiempo real" \  
  --metric-name Errors \  
  --namespace AWS/Lambda \  
  --statistic Sum \  
  --dimensions Name=FunctionName,Value=fn-rt-inference \  
  --period 300 \  
  --evaluation-periods 1 \  
  --threshold 2.0 \  
  --comparison-operator GreaterThanThreshold \  
  --treat-missing-data breaching
```

`--period 300` : Evalúa bloques de tiempo de 5 minutos.

`--threshold 2.0` : Si ocurren más de 2 ejecuciones fallidas (excepciones no controladas), el estado de la alarma cambia a `ALARM` .

`--treat-missing-data breaching` : Si el pipeline deja de emitir métricas abruptamente, se asume como un fallo de seguridad o caída del servicio.

3. Estrategia de Mantenimiento: Retraining del Modelo en SageMaker

Los patrones de ataque en redes corporativas cambian constantemente (degradación del modelo o *concept drift*). Para asegurar que el clasificador XGBoost mantenga su precisión a lo largo del tiempo, se debe implementar una ventana de reentrenamiento periódico:

1. **Recolección Automatizada:** El repositorio analítico en S3 (`raw-ingest/`) acumula diariamente los logs crudos preprocesados. [PDF](#)
2. **Etiquetado Forense:** Mensualmente, un analista de seguridad valida las IPs sospechosas en DynamoDB, consolidando un nuevo dataset maestro etiquetado (`is_anomaly`). [PDF](#)
3. **Disparador de Reentrenamiento:** Se configura una regla en Amazon **EventBridge** (ej. el primer domingo de cada mes) para ejecutar una función Lambda que automatice el lanzamiento del script `train_deploy.py` pasándole las nuevas rutas del Data Lake. [PDF](#)
4. **Actualización Ininterrumpida (Blue/Green):** SageMaker actualiza el endpoint `ep-network-anomaly` de manera transparente reemplazando el contenedor

antiguo sin perder peticiones ni generar tiempo de inactividad para las Lambdas en producción.

Con esto concluimos los aspectos técnicos de diseño, desarrollo, despliegue y monitoreo de la arquitectura de detección de anomalías. Quedo a tu disposición si deseas profundizar en algún componente específico o revisar las conclusiones de la solución. PDF